



## Programação Orientada a Objetos – Módulo 3 – Professora Luciene Motta – 2026.1

### Trabalho para nota – 2º Bimestre

**Organização:** Três alunos por grupo (máximo 3 e mínimo 2)

**Valor total:** 50 pontos

**Desenvolvimento:** O trabalho será desenvolvido em 3 aulas (2 tempos cada), mais 2 aulas para ajustes finais/envio (1 tempo) e o restante para as apresentações (3 tempos).

**Entrega:** “Sistema” implementado em PHP OO com formulário para entrada de dados.

**Apresentação:** Demonstração do problema e funcionamento do sistema.

### Orientações gerais

Cada trio deverá desenvolver um “pequeno sistema” em PHP Orientado a Objetos, utilizando como base um dos diagramas de classes propostos, conforme divisão em aula. O sistema deverá conter:

- Implementação de todas as classes, considerando as boas práticas. Todas devem conter:
  - Atributos encapsulados;
  - Método construtor;
  - Métodos get e set (com validação se necessário);
  - Métodos próprios de cada classe;
  - Sobrescrita de método (emprego de polimorfismo);
- Formulário HTML para entrada de dados (com estilização);
- Arquivo PHP para processar os dados, instanciar objetos e exibir os resultados (a exibição deve ser formatada, seguindo a estética do formulário de entrada de dados. Deve incluir um botão para retornar ao formulário inicial).

A superclasse não precisa ser abstrata, mas o grupo deverá garantir que ela seja utilizada corretamente pelas subclasses.

Cada grupo deverá também preparar uma breve apresentação para explicar o problema trabalhado, apresentar o diagrama de classes utilizado e demonstrar o funcionamento do sistema implementado.

#### Problema 1: Sistema de Monitoramento de Áreas de Risco

Com o aumento de situações relacionadas a enchentes, deslizamentos e períodos de seca, torna-se importante pensar em soluções tecnológicas que auxiliem o monitoramento de áreas de risco. A Defesa Civil de uma cidade deseja criar um sistema para registrar regiões vulneráveis, emitir alertas e apoiar ações preventivas junto à população.

O trio deverá desenvolver um sistema em PHP OO para representar diferentes tipos de áreas de risco e suas formas específicas de alerta.

```
AreaRisco
# nomeRegiao
# nivelRisco
# quantidadeMoradores
+ exibirDados()
+ emitirAlerta()
```



```
AreaEnchente extends AreaRisco
```

```
- nivelRio  
+ emitirAlerta()  
+ calcularVolumeAgua()  
+ evacuarFamilias()
```

```
AreaDeslizamento extends AreaRisco
```

```
- inclinacaoTerreno  
+ emitirAlerta()  
+ verificarSolo()  
+ interditarArea()
```

```
AreaSeca extends AreaRisco
```

```
- indiceUmidade  
+ emitirAlerta()  
+ calcularReservatorio()  
+ distribuirAgua()
```

### **Método polimórfico: emitirAlerta()**

Cada subclasse deverá implementar esse método de forma diferente, de acordo com o tipo de risco.

### **Problema 2: Sistema de Transparência das Eleições 2026 no Rio de Janeiro**

Em períodos eleitorais, a população precisa acessar informações claras, organizadas e confiáveis sobre candidatos, propostas e atuação pública. Pensando nas Eleições 2026, uma organização da sociedade civil deseja desenvolver um sistema educativo para registrar dados públicos de diferentes tipos de candidatos e estimular a cidadania, a transparência e o voto consciente.

O sistema não deverá defender partidos ou candidatos. O objetivo é apenas organizar informações e simular análises de dados.

```
Candidato
```

```
# nome  
# partido  
# quantidadeVotos  
+ exibirDados()  
+ calcularIndicePopularidade()
```

```
CandidatoGovernador extends Candidato
```

```
- municipiosVisitados  
+ calcularIndicePopularidade()  
+ apresentarPlanoSeguranca()  
+ calcularImpactoEstadual()
```

CandidatoDeputadoFederal extends Candidato

- projetosFederais
- + calcularIndicePopularidade()
- + registrarProjetoNacional()
- + consultarAtividadeLegislativa()

CandidatoDeputadoEstadual extends Candidato

- projetosEstaduais
- + calcularIndicePopularidade()
- + registrarProjetoEstadual()
- + calcularAtuacaoRegional()

**Método polimórfico:** calcularIndicePopularidade()

Cada subclasse deverá calcular o índice com base em critérios diferentes.

### Problema 3: Sistema de Cooperativas de Reciclagem

Cooperativas de reciclagem realizam um trabalho essencial para a preservação ambiental, geração de renda e redução do descarte inadequado de resíduos. Porém, muitas cooperativas enfrentam dificuldades para controlar os materiais recebidos, calcular valores de venda e organizar a produção.

O trio deverá desenvolver um sistema em PHP OO para representar diferentes materiais recicláveis e calcular seu valor de venda.

MaterialReciclavel

- # descricao
- # peso
- # valorQuilo
- + exibirDados()
- + calcularValorVenda()

Papel extends MaterialReciclavel

- tipoPapel
- + calcularValorVenda()
- + prensarMaterial()
- + separarImpurezas()

Plastico extends MaterialReciclavel

- tipoPlastico
- + calcularValorVenda()
- + triturarMaterial()
- + verificarReciclabilidade()

Metal extends MaterialReciclavel

- tipoMetal
- + calcularValorVenda()
- + fundirMaterial()
- + calcularPureza()



**Método polimórfico:** calcularValorVenda()

Cada material deverá calcular o valor final considerando regras específicas.

#### Problema 4: Sistema de Cursos

Uma instituição de ensino deseja organizar seus cursos em diferentes modalidades: presencial, online e híbrida. Cada modalidade possui regras próprias de funcionamento, acompanhamento e certificação.

O trio deverá desenvolver um sistema em PHP OO para cadastrar cursos e calcular a certificação dos estudantes de acordo com a modalidade escolhida.

```
Curso
# titulo
# cargaHoraria
# valor
+ exibirDados()
+ calcularCertificacao()

CursoPresencial extends Curso
- sala
+ calcularCertificacao()
+ registrarPresenca()
+ reservarLaboratorio()

CursoOnline extends Curso
- plataforma
+ calcularCertificacao()
+ liberarAulaGravada()
+ verificarProgressoOnline()

CursoHibrido extends Curso
- encontrosPresenciais
+ calcularCertificacao()
+ agendarEncontro()
+ integrarAtividadesOnline()
```

**Método polimórfico:** calcularCertificacao()

Cada modalidade deverá ter uma regra própria para certificação.

#### Problema 5: Sistema de Transportes

Uma empresa de mobilidade urbana deseja controlar diferentes tipos de transporte utilizados em seus serviços. Cada transporte possui características próprias e formas diferentes de calcular o custo da viagem.

O trio deverá desenvolver um sistema em PHP OO para representar ônibus, vans e carros de aplicativo.



```
Transporte
# modelo
# placa
# capacidade
+ exibirDados()
+ calcularCustoViagem()

Onibus extends Transporte
- quantidadeEixos
+ calcularCustoViagem()
+ calcularLotacao()
+ registrarParada()

Van extends Transporte
- possuiArCondicionado
+ calcularCustoViagem()
+ organizarRoteiro()
+ calcularBagagens()

CarroAplicativo extends Transporte
- categoria
+ calcularCustoViagem()
+ calcularTarifaDinamica()
+ avaliarMotorista()
```

**Método polimórfico:** `calcularCustoViagem()`

Cada tipo de transporte deverá calcular o custo da viagem de forma diferente.

### Problema 6: Sistema de Projetos Sociais

Uma organização social deseja acompanhar diferentes projetos voltados para educação, alimentação e saúde. Cada projeto possui uma forma diferente de medir impacto e organizar suas ações.

O trio deverá desenvolver um sistema em PHP OO para registrar projetos sociais e calcular seu impacto na comunidade.

```
ProjetoSocial
# nome
# publicoAlvo
# verba
+ exibirDados()
+ calcularImpacto()

ProjetoEducacao extends ProjetoSocial
- quantidadeAlunos
+ calcularImpacto()
+ organizarOficina()
+ emitirCertificados()
```

```
ProjetoAlimentacao extends ProjetoSocial
```

```
- quantidadeRefeicoes  
+ calcularImpacto()  
+ distribuirCestas()  
+ controlarEstoque()
```

```
ProjetoSaude extends ProjetoSocial
```

```
- quantidadeAtendimentos  
+ calcularImpacto()  
+ agendarConsulta()  
+ registrarTriagem()
```

**Método polimórfico:** `calcularImpacto()`

Cada projeto deverá calcular o impacto social com base em critérios próprios.

### Problema 7: Sistema de Eventos

Uma empresa responsável por eventos culturais e educacionais deseja organizar diferentes tipos de eventos, como shows, palestras e oficinas. Cada evento possui regras próprias de organização e cálculo de receita.

O trio deverá desenvolver um sistema em PHP OO para cadastrar eventos e calcular sua receita.

```
Evento
```

```
# nome  
# data  
# valorIngresso  
+ exibirDados()  
+ calcularReceita()
```

```
Show extends Evento
```

```
- artista  
+ calcularReceita()  
+ montarSetlist()  
+ reservarCamarim()
```

```
Palestra extends Evento
```

```
- palestrante  
+ calcularReceita()  
+ emitirCertificado()  
+ registrarPerguntaPublico()
```

```
Oficina extends Evento
```

```
- materialNecessario  
+ calcularReceita()  
+ organizarMateriais()  
+ avaliarParticipante()
```



**Método polimórfico:** calcularReceita()

Cada tipo de evento deverá calcular a receita de forma diferente.

---

## Organização das aulas - Cronograma de desenvolvimento do trabalho

### Aula 1 — Planejamento e implementação da superclasse

#### Objetivos da aula:

- Organização inicial do trabalho em trio;
- Definição das responsabilidades de cada integrante;
- Criação da estrutura de pastas e arquivos;
- Definição textual de todos os métodos das classes (exceto métodos básicos);
- Implementação completa da superclasse;
- Implementação dos métodos da superclasse (ou, pelo menos, os básicos).

#### O grupo deverá apresentar ao final da aula:

- Estrutura inicial organizada;
- Superclasse implementada;
- Atributos encapsulados;
- Construtor implementado;
- Métodos getters e setters implementados;
- Demais métodos da superclasse funcionando → Caso este item fique incompleto, deve ser concluído como tarefa de casa;
- Documento ou anotações contendo a descrição dos métodos das subclasses.

### Aula 2 — Implementação das subclasses e criação de objetos

#### Objetivos da aula:

- Implementação completa das subclasses;
- Implementação dos métodos próprios;
- Aplicação do polimorfismo;
- Criação de objetos para testes;
- Testes de funcionamento sem formulário.

#### O grupo deverá apresentar ao final da aula:

- Três subclasses implementadas;
- Métodos sobrescritos funcionando;
- Métodos específicos funcionando;
- Objetos criados manualmente em PHP;
- Testes realizados utilizando echo ou var\_dump().





## Aula 3 — Formulários HTML e integração com PHP

### Objetivos da aula:

- Criação dos formulários HTML;
- Envio dos dados para o PHP;
- Integração formulário + classes;
- Testagem geral do sistema;
- Ajustes iniciais de layout e saída de dados.

### O grupo deverá apresentar ao final da aula:

- Formulário funcionando;
- Dados enviados corretamente;
- Objetos criados a partir do formulário;
- Exibição organizada das informações;
- Sistema parcialmente finalizado.

## Aula 4 — Finalização e início das apresentações

### 1º Tempo — Ajustes finais e envio no AVA

#### Objetivos:

- Realizar testes finais;
- Corrigir erros encontrados;
- Organizar arquivos;
- Enviar o trabalho no AVA.

#### O grupo deverá enviar no AVA:

- Arquivos PHP;
- Arquivos HTML;
- Apresentação que será utilizada pelo grupo.

#### Toda apresentação deverá conter:

- problema trabalhado;
- explicação do sistema;
- diagrama de classes (sugiro usar representação gráfica);
- explicação dos métodos;
- demonstração do funcionamento.

### 2º Tempo — Início das apresentações

Os grupos iniciarão as apresentações dos sistemas desenvolvidos.

## Aula 5 — Continuação das apresentações

Continuação das apresentações dos grupos.